

quantiumtask1

November 6, 2024

Quantium Virtual Internship - Retail Strategy and Analytics by Mauricio Razon

Loading in Packages

```
[59]: import pandas as pd
import numpy as np
```

Load in CSV Data Sets

```
[60]: file_path = ""
transaction_data = pd.read_csv(file_path + "QVI_transaction_data.csv")
customer_data = pd.read_csv(file_path + "QVI_purchase_behaviour.csv")
```

0.1 Exploratory data analysis

The first step in any analysis is to first understand the data. Let's take a look at each of the datasets provided.

0.1.1 Examining transaction data

```
[61]: transaction_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   DATE                  264836 non-null  int64  
1   STORE_NBR             264836 non-null  int64  
2   LYLTY_CARD_NBR        264836 non-null  int64  
3   TXN_ID                264836 non-null  int64  
4   PROD_NBR              264836 non-null  int64  
5   PROD_NAME             264836 non-null  object  
6   PROD_QTY              264836 non-null  int64  
7   TOT_SALES             264836 non-null  float64
dtypes: float64(1), int64(6), object(1)
memory usage: 16.2+ MB
```

```
[62]: #First 10 rows of transaction data set
transaction_data.head(10)
```

```
[62]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	\
0	43390	1	1000	1	5	
1	43599	1	1307	348	66	
2	43605	1	1343	383	61	
3	43329	2	2373	974	69	
4	43330	2	2426	1038	108	
5	43604	4	4074	2982	57	
6	43601	4	4149	3333	16	
7	43601	4	4196	3539	24	
8	43332	5	5026	4525	42	
9	43330	7	7150	6900	52	

	PROD_NAME	PROD_QTY	TOT_SALES
0	Natural Chip Compny SeaSalt175g	2	6.0
1	CCs Nacho Cheese 175g	3	6.3
2	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8
5	Old El Paso Salsa Dip Tomato Mild 300g	1	5.1
6	Smiths Crinkle Chips Salt & Vinegar 330g	1	5.7
7	Grain Waves Sweet Chilli 210g	1	3.6
8	Doritos Corn Chip Mexican Jalapeno 150g	1	3.9
9	Grain Waves Sour Cream&Chives 210G	2	7.2

Examining customer data

```
[63]: customer_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
0   LYLTY_CARD_NBR        72637 non-null  int64
1   LIFESTAGE              72637 non-null  object
2   PREMIUM_CUSTOMER      72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB
```

```
[64]: #first 10 rows of customer data
customer_data.head(10)
```

```
[64]:
```

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream

2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream
5	1007	YOUNG SINGLES/COUPLES	Budget
6	1009	NEW FAMILIES	Premium
7	1010	YOUNG SINGLES/COUPLES	Mainstream
8	1011	OLDER SINGLES/COUPLES	Mainstream
9	1012	OLDER FAMILIES	Mainstream

Let's check if the columns we would expect to be numeric are in numeric form and date columns are in date format for the transaction data set.

```
[65]: transaction_data['DATE']
```

```
[65]: 0      43390
      1      43599
      2      43605
      3      43329
      4      43330
      ...
      264831  43533
      264832  43325
      264833  43410
      264834  43461
      264835  43365
      Name: DATE, Length: 264836, dtype: int64
```

```
[66]: transaction_data.dtypes
```

```
[66]: DATE                int64
      STORE_NBR           int64
      LYLTY_CARD_NBR      int64
      TXN_ID              int64
      PROD_NBR            int64
      PROD_NAME           object
      PROD_QTY            int64
      TOT_SALES           float64
      dtype: object
```

Clearly, we can see that the date column is not in the appropriate format. Let's change this to a date format.

```
[67]: ##### Convert DATE column to a date format
      #### A quick search online tells us that CSV and Excel integer dates begin on
      ↪ 30 Dec 1899
      origin_date = pd.to_datetime("1899-12-30")
      transaction_data['DATE'] = pd.to_datetime(transaction_data['DATE'],
      ↪ origin='1899-12-30', unit='D')
```

```
transaction_data['DATE']
```

```
[67]: 0      2018-10-17
      1      2019-05-14
      2      2019-05-20
      3      2018-08-17
      4      2018-08-18
      ...
      264831 2019-03-09
      264832 2018-08-13
      264833 2018-11-06
      264834 2018-12-27
      264835 2018-09-22
      Name: DATE, Length: 264836, dtype: datetime64[ns]
```

We should check that we are looking at the right products by examining PROD_NAME.

```
[68]: transaction_data['PROD_NAME'].value_counts()
```

```
[68]: PROD_NAME
Kettle Mozzarella Basil & Pesto 175g      3304
Kettle Tortilla ChpsHny&Jlpno Chili 150g   3296
Cobs Popd Swt/Chlli &Sr/Cream Chips 110g     3269
Tyrrells Crisps Ched & Chives 165g         3268
Cobs Popd Sea Salt Chips 110g               3265
...
RRD Pc Sea Salt 165g                       1431
Woolworths Medium Salsa 300g               1430
NCC Sour Cream & Garden Chives 175g        1419
French Fries Potato Chips 175g              1418
WW Crinkle Cut Original 175g                1410
Name: count, Length: 114, dtype: int64
```

Looks like we are definitely looking at potato chips but how can we check that these are all chips?
We can do some basic text analysis by summarising the individual words in the product name.

```
[69]: #### Examine the words in PROD_NAME to see if there are any incorrect entries
      #### such as products that are not chips
product_words = transaction_data['PROD_NAME'].unique() # Get unique product_
↳ names
words_list = [word for prod in product_words for word in prod.split()]
product_words_df = pd.DataFrame(words_list, columns=['words'])
product_words_df
```

```
[69]:      words
0    Natural
1      Chip
2    Compny
```

```

3    SeaSalt175g
4          CCs
..          ...
584         150g
585       Doritos
586         Salsa
587         Mild
588         300g

```

```
[589 rows x 1 columns]
```

As we are only interested in words that will tell us if the product is chips or not, let's remove all words with digits and special characters such as '&' from our set of product words.

```
[70]: valid_words = product_words_df['words'].str.contains(r'^[A-Za-z\s]+$',
↳ regex=True)
valid_words_df = product_words_df[valid_words]
word_counts = valid_words_df['words'].value_counts()
sorted_word_counts = word_counts.sort_values(ascending=False)
print(sorted_word_counts)
```

```

words
Chips      21
Smiths     16
Crinkle    14
Cut         14
Kettle     13
..
Vingar      1
Veg         1
Strws       1
Mango       1
Bolognese   1
Name: count, Length: 168, dtype: int64

```

There are salsa products in the dataset but we are only interested in the chips category, so let's remove these.

```
[71]: ##### Remove salsa products
transaction_data['SALSA'] = transaction_data['PROD_NAME'].str.contains('salsa',
↳ case=False, na=False)
transaction_data = transaction_data[~transaction_data['SALSA']]
transaction_data = transaction_data.drop(columns=['SALSA'])
transaction_data
```

```
[71]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	\
0	2018-10-17	1	1000	1	5	
1	2019-05-14	1	1307	348	66	

2	2019-05-20	1	1343	383	61
3	2018-08-17	2	2373	974	69
4	2018-08-18	2	2426	1038	108
...	...	...	...	...	...
264831	2019-03-09	272	272319	270088	89
264832	2018-08-13	272	272358	270154	74
264833	2018-11-06	272	272379	270187	51
264834	2018-12-27	272	272379	270188	42
264835	2018-09-22	272	272380	270189	74

		PROD_NAME	PROD_QTY	TOT_SALES
0	Natural Chip	Compny SeaSalt175g	2	6.0
1		CCs Nacho Cheese 175g	3	6.3
2	Smiths Crinkle Cut	Chips Chicken 170g	2	2.9
3	Smiths Chip Thinly	S/Cream&Onion 175g	5	15.0
4	Kettle Tortilla ChpsHny&Jlpno	Chili 150g	3	13.8
...	...	...	...	...
264831	Kettle Sweet Chillli And Sour Cream	175g	2	10.8
264832	Tostitos Splash Of	Lime 175g	1	4.4
264833	Doritos Mexicana	170g	2	8.8
264834	Doritos Corn Chip Mexican	Jalapeno 150g	2	7.8
264835	Tostitos Splash Of	Lime 175g	2	8.8

[246742 rows x 8 columns]

Next, we generate a summary to check statistics such as mean, min and max values for each feature to see if there are any obvious outliers in the data and if there are any nulls in any of the columns (NA's : number of nulls will appear in the output if there are any nulls)

```
[72]: transaction_data.describe()
```

```
[72]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	\
count	246742	246742.000000	2.467420e+05	
mean	2018-12-30 01:19:01.211467520	135.051098	1.355310e+05	
min	2018-07-01 00:00:00	1.000000	1.000000e+03	
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	
75%	2019-03-31 00:00:00	203.000000	2.030840e+05	
max	2019-06-30 00:00:00	272.000000	2.373711e+06	
std	NaN	76.787096	8.071528e+04	

	TXN_ID	PROD_NBR	PROD_QTY	TOT_SALES
count	2.467420e+05	246742.000000	246742.000000	246742.000000
mean	1.351311e+05	56.351789	1.908062	7.321322
min	1.000000e+00	1.000000	1.000000	1.700000
25%	6.756925e+04	26.000000	2.000000	5.800000
50%	1.351830e+05	53.000000	2.000000	7.400000

75%	2.026538e+05	87.000000	2.000000	8.800000
max	2.415841e+06	114.000000	200.000000	650.000000
std	7.814772e+04	33.695428	0.659831	3.077828

```
[73]: null_counts = transaction_data.isnull().sum()
null_counts
```

```
[73]: DATE          0
STORE_NBR        0
LYLTY_CARD_NBR   0
TXN_ID           0
PROD_NBR         0
PROD_NAME        0
PROD_QTY         0
TOT_SALES        0
dtype: int64
```

There are no nulls in the columns but product quantity appears to have an outlier which we should investigate further. Let's investigate further the case where 200 packets of chips are bought in one transaction.

```
[74]: ##### Filter the dataset to find the outlier
outlier_transactions = transaction_data[transaction_data['PROD_QTY'] == 200]
outlier_transactions
```

```
[74]:          DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
69762 2018-08-19        226        226000  226201         4
69763 2019-05-20        226        226000  226210         4

          PROD_NAME  PROD_QTY  TOT_SALES
69762  Dorito Corn Chp    Supreme 380g      200      650.0
69763  Dorito Corn Chp    Supreme 380g      200      650.0
```

There are two transactions where 200 packets of chips are bought in one transaction and both of these transactions were by the same customer.

```
[75]: ##### Let's see if the customer has had other transactions
loyalty_card_number = outlier_transactions['LYLTY_CARD_NBR'].values[0]
customer_transactions = transaction_data[transaction_data['LYLTY_CARD_NBR'] ==
↳ loyalty_card_number]
customer_transactions
```

```
[75]:          DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
69762 2018-08-19        226        226000  226201         4
69763 2019-05-20        226        226000  226210         4

          PROD_NAME  PROD_QTY  TOT_SALES
69762  Dorito Corn Chp    Supreme 380g      200      650.0
```

69763	Dorito Corn Chp	Supreme 380g	200	650.0
-------	-----------------	--------------	-----	-------

It looks like this customer has only had the two transactions over the year and is not an ordinary retail customer. The customer might be buying chips for commercial purposes instead. We'll remove this loyalty card number from further analysis.

```
[76]: transaction_data = transaction_data[transaction_data['LYLTY_CARD_NBR'] !=
↳loyalty_card_number]
transaction_data
```

```
[76]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	\
0	2018-10-17	1	1000	1	5	
1	2019-05-14	1	1307	348	66	
2	2019-05-20	1	1343	383	61	
3	2018-08-17	2	2373	974	69	
4	2018-08-18	2	2426	1038	108	
...	...	...	...	...	...	
264831	2019-03-09	272	272319	270088	89	
264832	2018-08-13	272	272358	270154	74	
264833	2018-11-06	272	272379	270187	51	
264834	2018-12-27	272	272379	270188	42	
264835	2018-09-22	272	272380	270189	74	

	PROD_NAME	PROD_QTY	TOT_SALES
0	Natural Chip Compny SeaSalt175g	2	6.0
1	CCs Nacho Cheese 175g	3	6.3
2	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8
...	...	...	...
264831	Kettle Sweet Chillli And Sour Cream 175g	2	10.8
264832	Tostitos Splash Of Lime 175g	1	4.4
264833	Doritos Mexicana 170g	2	8.8
264834	Doritos Corn Chip Mexican Jalapeno 150g	2	7.8
264835	Tostitos Splash Of Lime 175g	2	8.8

[246740 rows x 8 columns]

```
[77]: transaction_data.describe()
```

```
[77]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	\
count	246740	246740.000000	2.467400e+05	
mean	2018-12-30 01:18:58.448569344	135.050361	1.355303e+05	
min	2018-07-01 00:00:00	1.000000	1.000000e+03	
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	
75%	2019-03-31 00:00:00	203.000000	2.030832e+05	

max	2019-06-30 00:00:00	272.000000	2.373711e+06
std	NaN	76.786971	8.071520e+04

	TXN_ID	PROD_NBR	PROD_QTY	TOT_SALES
count	2.467400e+05	246740.000000	246740.000000	246740.000000
mean	1.351304e+05	56.352213	1.906456	7.316113
min	1.000000e+00	1.000000	1.000000	1.700000
25%	6.756875e+04	26.000000	2.000000	5.800000
50%	1.351815e+05	53.000000	2.000000	7.400000
75%	2.026522e+05	87.000000	2.000000	8.800000
max	2.415841e+06	114.000000	5.000000	29.500000
std	7.814760e+04	33.695235	0.342499	2.474897

That's better. Now, let's look at the number of transaction lines over time to see if there are any obvious data issues such as missing data.

```
[78]: ##### Count the number of transactions by date
transaction_count_by_date = transaction_data.groupby('DATE').size().
    ↪reset_index(name='TRANSACTION_COUNT')
transaction_count_by_date.sort_values(by = 'TRANSACTION_COUNT', ascending =
    ↪False)
```

```
[78]:      DATE  TRANSACTION_COUNT
176 2018-12-24                865
175 2018-12-23                853
174 2018-12-22                840
171 2018-12-19                839
172 2018-12-20                808
..      ...
357 2019-06-24                612
109 2018-10-18                611
147 2018-11-25                610
83  2018-09-22                609
346 2019-06-13                607
```

[364 rows x 2 columns]

There's only 364 rows, meaning only 364 dates which indicates a missing date. Let's create a sequence of dates from 1 Jul 2018 to 30 Jun 2019 and use this to create a chart of number of transactions over time to find the missing date.

```
[79]: ##### Create a sequence of dates and join this the count of transactions by date
date_range = pd.date_range(start='2018-07-01', end='2019-06-30')
date_df = pd.DataFrame(date_range, columns=['DATE'])
transaction_data.loc[:, 'DATE'] = pd.to_datetime(transaction_data['DATE'])
transaction_count = transaction_data.groupby('DATE').size().
    ↪reset_index(name='TRANSACTION_COUNT')
```

```
merged_data = pd.merge(date_df, transaction_count, on='DATE', how='left')
merged_data.loc[:, 'TRANSACTION_COUNT'] = merged_data['TRANSACTION_COUNT'].
    ↪fillna(0)
merged_data = pd.merge(merged_data, transaction_data, on='DATE', how='left')
merged_data
```

```
[79]:
```

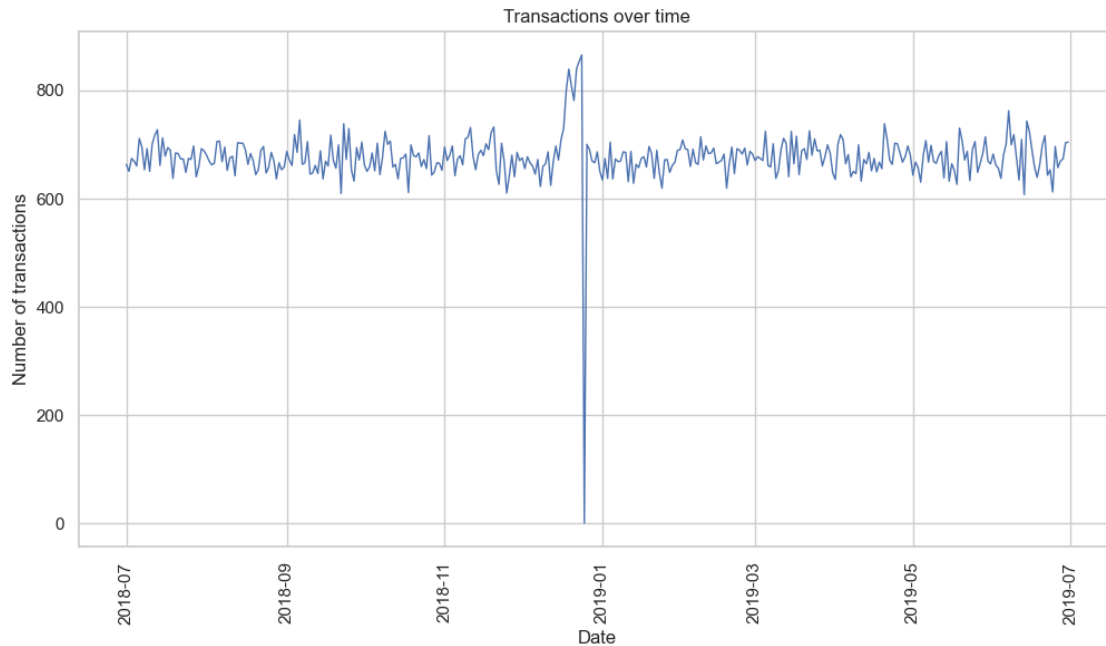
	DATE	TRANSACTION_COUNT	STORE_NBR	LYLTY_CARD_NBR	TXN_ID \
0	2018-07-01	663.0	47.0	47142.0	42540.0
1	2018-07-01	663.0	55.0	55073.0	48884.0
2	2018-07-01	663.0	55.0	55073.0	48884.0
3	2018-07-01	663.0	58.0	58351.0	54374.0
4	2018-07-01	663.0	68.0	68193.0	65598.0
...	...	...	...	...	...
246736	2019-06-30	704.0	242.0	242159.0	246222.0
246737	2019-06-30	704.0	244.0	244213.0	247339.0
246738	2019-06-30	704.0	256.0	256018.0	255130.0
246739	2019-06-30	704.0	257.0	257079.0	256218.0
246740	2019-06-30	704.0	265.0	265006.0	263307.0

	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
0	14.0	Smiths Crnkle Chip Orgnl Big Bag 380g	2.0	11.8
1	99.0	Pringles Sthrn FriedChicken 134g	2.0	7.4
2	91.0	CCs Tasty Cheese 175g	2.0	4.2
3	102.0	Kettle Mozzarella Basil & Pesto 175g	2.0	10.8
4	44.0	Thins Chips Light& Tangy 175g	2.0	6.6
...	...	...	...	...
246736	36.0	Kettle Chillli 175g	2.0	10.8
246737	93.0	Doritos Corn Chip Southern Chicken 150g	2.0	7.8
246738	105.0	Woolworths Cheese Rings 190g	2.0	3.6
246739	71.0	Twisties Cheese Burger 250g	2.0	8.6
246740	106.0	Natural ChipCo Hony Soy Chckn175g	1.0	3.0

[246741 rows x 9 columns]

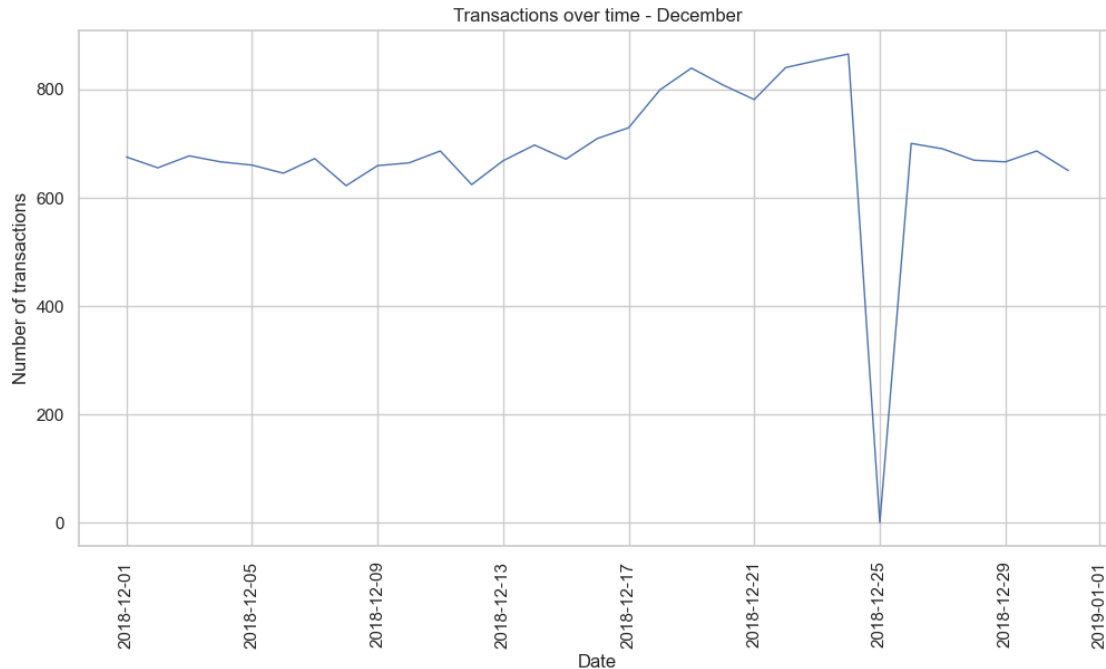
```
[80]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
all_dates = pd.DataFrame({'DATE': pd.date_range(start="2018-07-01",
    ↪end="2019-06-30")})
transaction_counts = transaction_data.groupby('DATE').size().
    ↪reset_index(name='count')
transactions_by_day = all_dates.merge(transaction_counts, on='DATE',
    ↪how='left').fillna(0)
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))
plt.plot(transactions_by_day['DATE'], transactions_by_day['count'], marker='.',
    ↪color='b', linewidth=1)
```

```
plt.xlabel("Date")
plt.ylabel("Number of transactions")
plt.title("Transactions over time")
plt.xticks(rotation=90)
plt.show()
```



We can see that there is an increase in purchases in December and a break in late December. Let's zoom in on this.

```
[81]: transactions_december = transactions_by_day[transactions_by_day['DATE'].dt.
      ↪month == 12]
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))
plt.plot(transactions_december['DATE'], transactions_december['count'],
      ↪marker='', color='b', linewidth=1)
plt.xlabel("Date")
plt.ylabel("Number of transactions")
plt.title("Transactions over time - December")
plt.xticks(rotation=90)
plt.show()
```



We can see that the increase in sales occurs in the lead-up to Christmas and that there are zero sales on Christmas day itself. This is due to shops being closed on Christmas day. Now that we are satisfied that the data no longer has outliers, we can move on to creating other features such as brand of chips or pack size from PROD_NAME. We will start with pack size.

```
[82]: ##### Pack size
##### We can work this out by taking the digits that are in PROD_NAME
transaction_data['PACK_SIZE'] = transaction_data['PROD_NAME'].str.
    .extract(r'(\d+)').astype(float)
pack_size_counts = transaction_data['PACK_SIZE'].value_counts().sort_index()
print(pack_size_counts)
```

```
PACK_SIZE
70.0      1507
90.0      3008
110.0     22387
125.0     1454
134.0     25102
135.0      3257
150.0     40203
160.0      2970
165.0     15297
170.0     19983
175.0     66390
180.0      1468
190.0      2995
```

```

200.0    4473
210.0    6272
220.0    1564
250.0    3169
270.0    6285
330.0   12540
380.0    6416
Name: count, dtype: int64

```

C:\Users\razon\AppData\Local\Temp\ipykernel_3512\2480084294.py:3:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

transaction_data['PACK_SIZE'] =
transaction_data['PROD_NAME'].str.extract(r'(\d+)').astype(float)

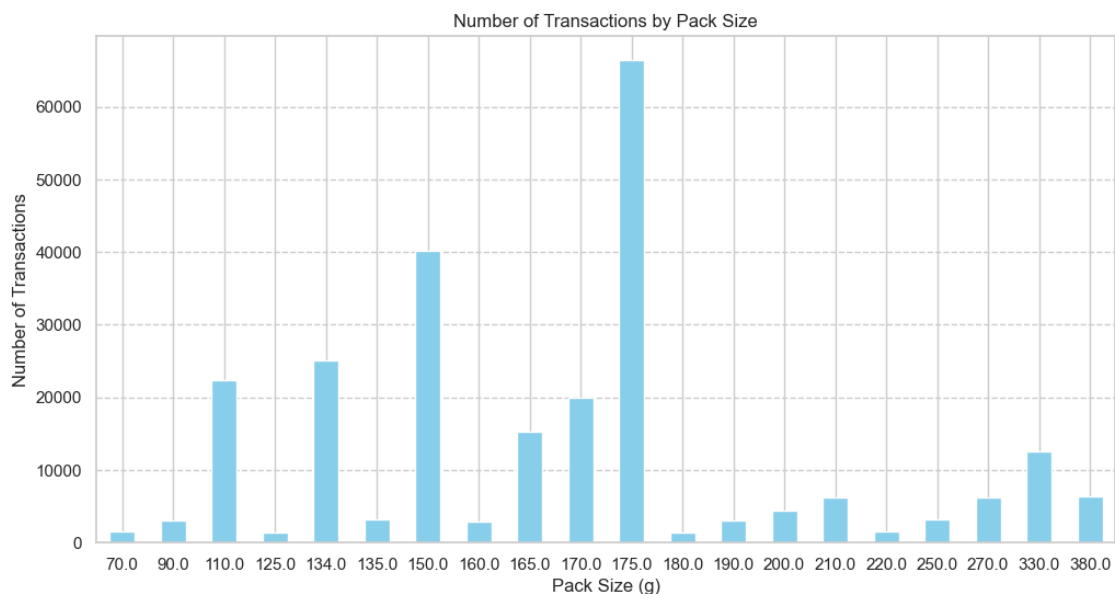
```

The largest size is 380g and the smallest size is 70g - seems sensible!

```

[84]: plt.figure(figsize=(12, 6))
transaction_data['PACK_SIZE'].value_counts().sort_index().plot(kind='bar',
↳ color='skyblue')
plt.title('Number of Transactions by Pack Size')
plt.xlabel('Pack Size (g)')
plt.ylabel('Number of Transactions')
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--')
plt.show()

```



Pack sizes look reasonable.

Now to create brands, we can use the first word in PROD_NAME to work out the brand name...

```
[89]: ##### Brands
transaction_data.loc[:, 'BRAND'] = transaction_data['PROD_NAME'].str.split().
    ↪str[0]
brand_counts = transaction_data['BRAND'].value_counts()
brand_counts
```

```
[89]: BRAND
Kettle          41288
Smiths          27390
Pringles       25102
Doritos        22041
Thins          14075
RRD            11894
Infuzions      11057
WW             10320
Cobs           9693
Tostitos       9471
Twisties       9454
Tyrrells       6442
Grain          6272
Natural        6050
Cheezels       4603
CCs            4551
Red            4427
Dorito         3183
Infzns         3144
Smith          2963
Cheetos        2927
Snbts          1576
Burger         1564
Woolworths     1516
GrnWves        1468
Sunbites       1432
NCC            1419
French         1418
Name: count, dtype: int64
```

Some of the brand names look like they are of the same brands - such as RED and RRD, which are both Red Rock Deli chips. Let's combine these together.

```
[98]: brand_corrections = {
      'Red': 'RRD',
      'Snbts': 'SUNBITES',
```

```

    'INFZNS': 'INFUZIONI',
    'WW': 'WOOLWORTHS',
    'Smith': 'SMITHS',
    'NCC': 'NATURAL',
    'DORITO': 'DORITOS',
    'Doritos': 'DORITOS',
    'Grain': 'GRNWVES',
    'Infzns': 'INFUZIONI',
    'Infuzioni': 'INFUZIONI',
    'Dorito': 'DORITOS',
    'Sunbites': 'SUNBITES',
    'GrnWves': 'GRNWVES',
    'Burger': 'BURGER',
    'Cheetos': 'CHEETOS',
    'Cobs': 'COBS',
    'Twisties': 'TWISTIES',
    'Tostitos': 'TOSTITOS',
    'Pringles': 'PRINGLES',
    'Kettle': 'KETTLE',
    'Smiths': 'SMITHS',
    'Tyrrells': 'TYRRELLS',
    'CCs': 'CCS',
    'Thins': 'THINS',
    'French': 'FRENCH',
    'Natural': 'NATURAL',
    'Woolworths': 'WOOLWORTHS',
    'Cheezels': 'CHEEZELS',
    'RRD': 'RRD',
    'Old': 'OLD'
}
transaction_data.loc[:, 'BRAND'] = transaction_data['BRAND'].
    ↪replace(brand_corrections)
transaction_data['BRAND'].value_counts()

```

```

[98]: BRAND
KETTLE      41288
SMITHS      30353
DORITOS     25224
PRINGLES    25102
RRD         16321
INFUZIONI   14201
THINS       14075
WOOLWORTHS  11836
COBS        9693
TOSTITOS    9471
TWISTIES    9454
GRNWVES     7740

```

```

NATURAL      7469
TYRRELLS     6442
CHEEZELS     4603
CCS          4551
SUNBITES     3008
CHEETOS      2927
BURGER       1564
FRENCH       1418
Name: count, dtype: int64

```

1 Examining customer data

Now that we are happy with the transaction dataset, let's have a look at the customer dataset. Basic summaries of the dataset, including distributions of any key columns

```
[99]: customer_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
0   LYLTY_CARD_NBR         72637 non-null  int64
1   LIFESTAGE              72637 non-null  object
2   PREMIUM_CUSTOMER      72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB

```

```
[100]: customer_data['LYLTY_CARD_NBR'].value_counts()
```

```

[100]: LYLTY_CARD_NBR
1000      1
181211    1
181217    1
181216    1
181215    1
..
88126     1
88127     1
88128     1
88129     1
2373711   1
Name: count, Length: 72637, dtype: int64

```

```
[101]: customer_data['LIFESTAGE'].value_counts()
```

```
[101]: LIFESTAGE
      RETIREES          14805
      OLDER SINGLES/COUPLES 14609
      YOUNG SINGLES/COUPLES 14441
      OLDER FAMILIES      9780
      YOUNG FAMILIES      9178
      MIDAGE SINGLES/COUPLES 7275
      NEW FAMILIES        2549
      Name: count, dtype: int64
```

```
[102]: customer_data['PREMIUM_CUSTOMER'].value_counts()
```

```
[102]: PREMIUM_CUSTOMER
      Mainstream    29245
      Budget        24470
      Premium       18922
      Name: count, dtype: int64
```

Merge transactions data to costumer data

```
[103]: merged_data = pd.merge(transaction_data, customer_data, on='LYLTY_CARD_NBR',
      ↪how='left')
      merged_data
```

```
[103]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	\
0	2018-10-17	1	1000	1	5	
1	2019-05-14	1	1307	348	66	
2	2019-05-20	1	1343	383	61	
3	2018-08-17	2	2373	974	69	
4	2018-08-18	2	2426	1038	108	
...	...	...	...	...	...	
246735	2019-03-09	272	272319	270088	89	
246736	2018-08-13	272	272358	270154	74	
246737	2018-11-06	272	272379	270187	51	
246738	2018-12-27	272	272379	270188	42	
246739	2018-09-22	272	272380	270189	74	

	PROD_NAME	PROD_QTY	TOT_SALES	\
0	Natural Chip Compny SeaSalt175g	2	6.0	
1	CCs Nacho Cheese 175g	3	6.3	
2	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	
3	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0	
4	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8	
...	...	...	...	
246735	Kettle Sweet Chillli And Sour Cream 175g	2	10.8	
246736	Tostitos Splash Of Lime 175g	1	4.4	
246737	Doritos Mexicana 170g	2	8.8	

246738	Doritos Corn Chip Mexican Jalapeno 150g	2	7.8
246739	Tostitos Splash Of Lime 175g	2	8.8

	PACK_SIZE	BRAND		LIFESTAGE	PREMIUM_CUSTOMER
0	175.0	NATURAL	YOUNG	SINGLES/COUPLES	Premium
1	175.0	CCS	MIDAGE	SINGLES/COUPLES	Budget
2	170.0	SMITHS	MIDAGE	SINGLES/COUPLES	Budget
3	175.0	SMITHS	MIDAGE	SINGLES/COUPLES	Budget
4	150.0	KETTLE	MIDAGE	SINGLES/COUPLES	Budget
...	...	...		...	...
246735	175.0	KETTLE	YOUNG	SINGLES/COUPLES	Premium
246736	175.0	TOSTITOS	YOUNG	SINGLES/COUPLES	Premium
246737	170.0	DORITOS	YOUNG	SINGLES/COUPLES	Premium
246738	150.0	DORITOS	YOUNG	SINGLES/COUPLES	Premium
246739	175.0	TOSTITOS	YOUNG	SINGLES/COUPLES	Premium

[246740 rows x 12 columns]

As the number of rows in `data` is the same as that of `transactionData`, we can be sure that no duplicates were created.

Let's also check if some customers were not matched on by checking for nulls

```
[104]: merged_data.isnull().any()
```

```
[104]: DATE                False
STORE_NBR                False
LYLTY_CARD_NBR          False
TXN_ID                   False
PROD_NBR                 False
PROD_NAME                False
PROD_QTY                 False
TOT_SALES                False
PACK_SIZE                False
BRAND                    False
LIFESTAGE                False
PREMIUM_CUSTOMER         False
dtype: bool
```

Great, there are no nulls! So all our customers in the transaction data has been accounted for in the customer dataset.

Converting merged data set for modeling and analysis

```
[105]: filePath = '/Users/razon/Downloads'
merged_data.to_csv(f"{filePath}QVI_data.csv", index=False)
```

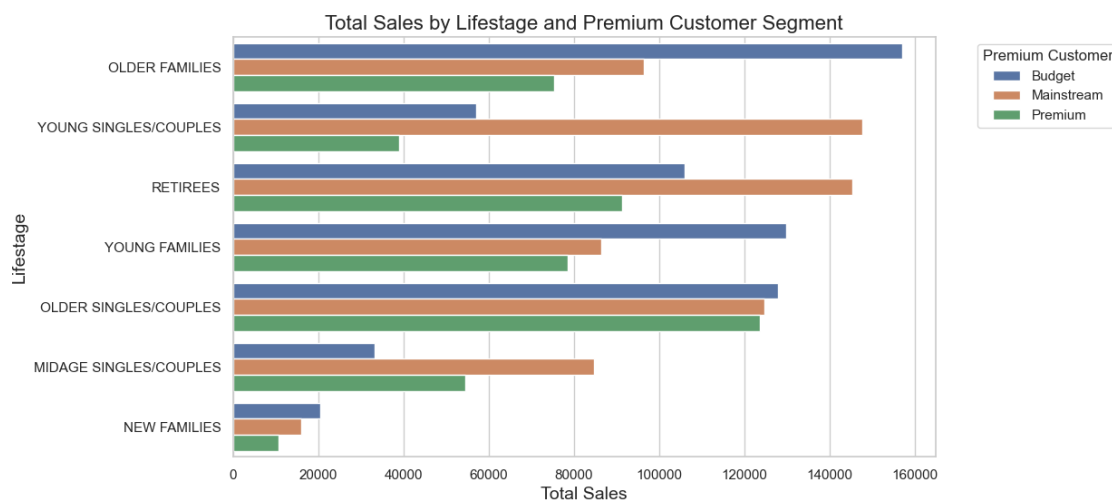
1.1 Data analysis on customer segments

Now that the data is ready for analysis, we can define some metrics of interest to the client: - Who spends the most on chips (total sales), describing customers by lifestage and how premium their general purchasing behaviour is - How many customers are in each segment- How many chips are bought per customer by segment - What's the average chip price by customer segment We could also ask our data team for more information.

Examples are: - The customer's total spend over the period and total spend for each transaction to understand what proportion of their grocery spend is on chips - Proportion of customers in each customer segment overall to compare against the mix of customers who purchase chips

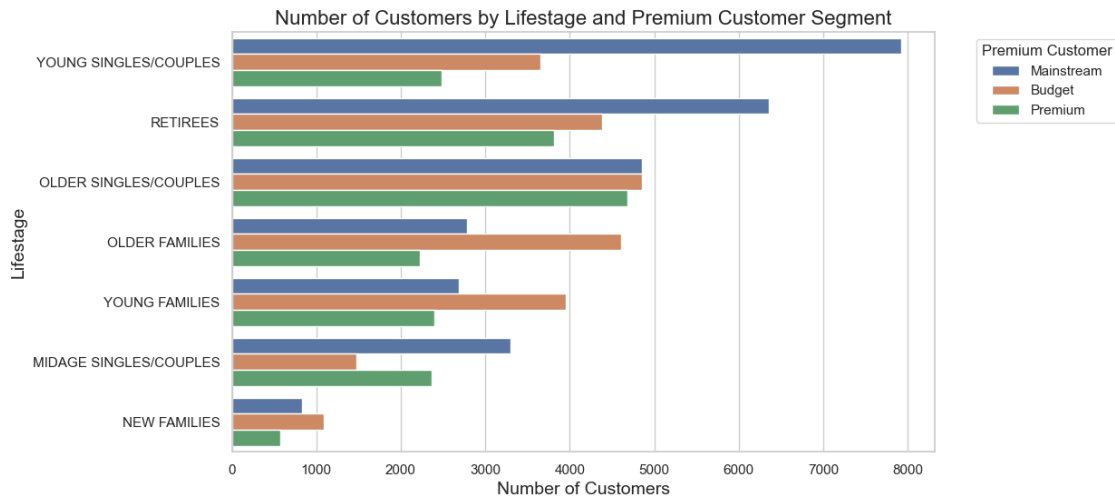
Let's start with calculating total sales by LIFESTAGE and PREMIUM_CUSTOMER and plotting the split by these segments to describe which customer segment contribute most to chip sales.

```
[106]: ##### Total sales by LIFESTAGE and PREMIUM_CUSTOMER
total_sales = merged_data.groupby(['LIFESTAGE',
    ↪ 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum().reset_index()
total_sales = total_sales.sort_values(by='TOT_SALES', ascending=False)
plt.figure(figsize=(10, 6))
sns.barplot(data=total_sales, x='TOT_SALES', y='LIFESTAGE', hue=
    ↪ 'PREMIUM_CUSTOMER', dodge=True)
plt.title('Total Sales by Lifestage and Premium Customer Segment', fontsize=16)
plt.xlabel('Total Sales', fontsize=14)
plt.ylabel('Lifestage', fontsize=14)
plt.legend(title='Premium Customer', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Sales are coming mainly from Budget - older families, Mainstream - young singles/couples, and Mainstream - retirees. Let's see if the higher sales are due to there being more customers who buy chips

```
[107]: ##### Number of customers by LIFESTAGE and PREMIUM_CUSTOMER
customer_count = merged_data.groupby(['LIFESTAGE',
    ↳ 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR'].nunique().reset_index()
customer_count.rename(columns={'LYLTY_CARD_NBR': 'NUM_CUSTOMERS'}, inplace=True)
customer_count = customer_count.sort_values(by='NUM_CUSTOMERS', ascending=False)
plt.figure(figsize=(10, 6))
sns.barplot(data=customer_count, x='NUM_CUSTOMERS', y='LIFESTAGE',
    ↳ hue='PREMIUM_CUSTOMER', dodge=True)
plt.title('Number of Customers by Lifestage and Premium Customer Segment',
    ↳ fontsize=16)
plt.xlabel('Number of Customers', fontsize=14)
plt.ylabel('Lifestage', fontsize=14)
plt.legend(title='Premium Customer', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



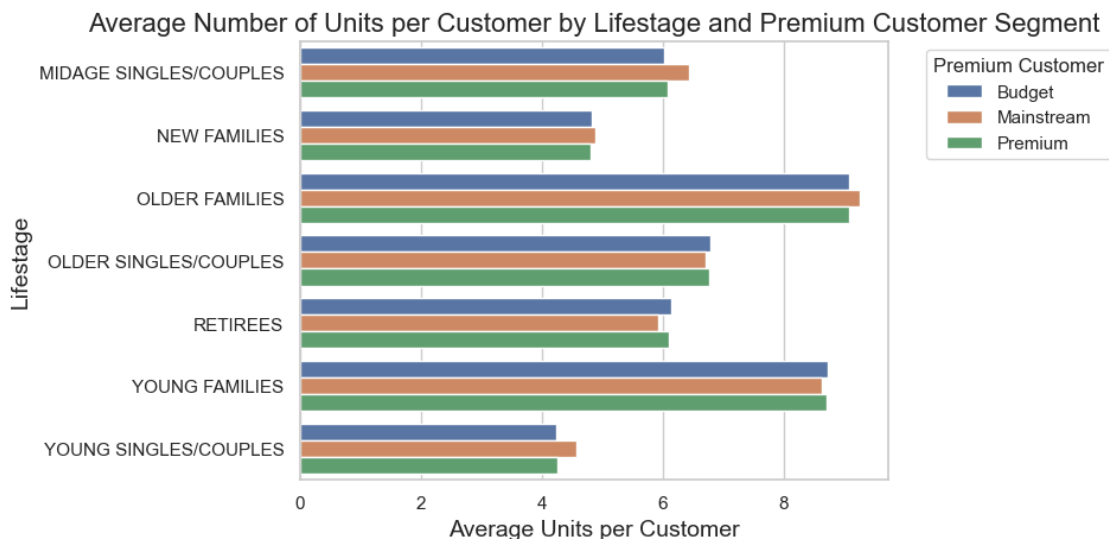
There are more Mainstream - young singles/couples and Mainstream - retirees who buy chips. This contributes to there being more sales to these customer segments but this is not a major driver for the Budget - Older families segment. Higher sales may also be driven by more units of chips being bought per customer. Let's have a look at this next.

```
[108]: ##### Average number of units per customer by LIFESTAGE and PREMIUM_CUSTOMER
total_sales = merged_data.groupby(['LIFESTAGE',
    ↳ 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum().reset_index()
customer_count = merged_data.groupby(['LIFESTAGE',
    ↳ 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR'].nunique().reset_index()
customer_count.rename(columns={'LYLTY_CARD_NBR': 'CUSTOMER_COUNT'},
    ↳ inplace=True)
units_per_customer = merged_data.groupby(['LYLTY_CARD_NBR', 'LIFESTAGE',
    ↳ 'PREMIUM_CUSTOMER'])['PROD_QTY'].sum().reset_index()
```

```

avg_units = units_per_customer.groupby(['LIFESTAGE',
    ↪ 'PREMIUM_CUSTOMER'])['PROD_QTY'].mean().reset_index()
avg_units.rename(columns={'PROD_QTY': 'AVG_UNITS'}, inplace=True)
merged_data['AVG_PRICE'] = merged_data['TOT_SALES'] / merged_data['PROD_QTY']
avg_price = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['AVG_PRICE'].
    ↪ mean().reset_index()
avg_price.rename(columns={'AVG_PRICE': 'AVG_CHIP_PRICE'}, inplace=True)
summary = total_sales.merge(customer_count, on=['LIFESTAGE',
    ↪ 'PREMIUM_CUSTOMER'])
summary = summary.merge(avg_units, on=['LIFESTAGE', 'PREMIUM_CUSTOMER'])
summary = summary.merge(avg_price, on=['LIFESTAGE', 'PREMIUM_CUSTOMER'])
sns.barplot(data=avg_units, x='AVG_UNITS', y='LIFESTAGE',
    ↪ hue='PREMIUM_CUSTOMER', dodge=True)
plt.title('Average Number of Units per Customer by Lifestage and Premium
    ↪ Customer Segment', fontsize=16)
plt.xlabel('Average Units per Customer', fontsize=14)
plt.ylabel('Lifestage', fontsize=14)
plt.legend(title='Premium Customer', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()

```



Older families and young families in general buy more chips per customer.

Let's also investigate the average price per unit chips bought for each customer segment as this is also a driver of total sales

```

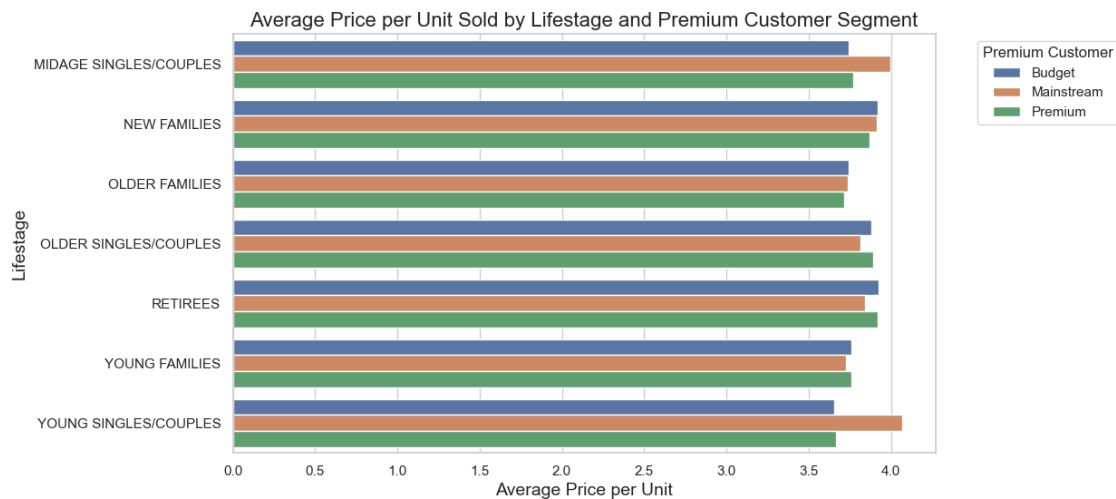
[109]: merged_data['AVG_PRICE_PER_UNIT'] = merged_data['TOT_SALES'] /
    ↪ merged_data['PROD_QTY']
avg_price_per_unit = merged_data.groupby(['LIFESTAGE',
    ↪ 'PREMIUM_CUSTOMER'])['AVG_PRICE_PER_UNIT'].mean().reset_index()

```

```

avg_price_per_unit.rename(columns={'AVG_PRICE_PER_UNIT': 'AVG_PRICE'},
    inplace=True)
plt.figure(figsize=(10, 6))
sns.barplot(data=avg_price_per_unit, x='AVG_PRICE', y='LIFESTAGE',
    hue='PREMIUM_CUSTOMER', dodge=True)
plt.title('Average Price per Unit Sold by Lifestage and Premium Customer
    Segment', fontsize=16)
plt.xlabel('Average Price per Unit', fontsize=14)
plt.ylabel('Lifestage', fontsize=14)
plt.legend(title='Premium Customer', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()

```



Mainstream midage and young singles and couples are more willing to pay more per packet of chips compared to their budget and premium counterparts.

This may be due to premium shoppers being more likely to buy healthy snacks and when they buy chips, this is mainly for entertainment purposes rather than their own consumption.

This is also supported by there being fewer premium midage and young singles and couples buying chips compared to their mainstream counterparts.

As the difference in average price per unit isn't large, we can check if this difference is statistically different.

```

[114]: #Independent T-test between mainstream vs premium and budget midage and young
    singles and couples
    from scipy.stats import ttest_ind
    merged_data['pricePerUnit'] = merged_data['TOT_SALES'] / merged_data['PROD_QTY']

    # Define the two groups
    mainstream_group = merged_data[(merged_data['LIFESTAGE'].isin(["YOUNG SINGLES/
    COUPLES", "MIDAGE SINGLES/COUPLES"])) &

```

```

        (merged_data['PREMIUM_CUSTOMER'] == '
↳Mainstream'))['pricePerUnit']
non_mainstream_group = merged_data[(merged_data['LIFESTAGE'].isin(['YOUNG_
↳SINGLES/COUPLES', 'MIDAGE SINGLES/COUPLES'])) &
        (merged_data['PREMIUM_CUSTOMER'] != '
↳Mainstream'))['pricePerUnit']

# Perform the t-test
t_stat, p_value = ttest_ind(mainstream_group, non_mainstream_group,
↳alternative='greater')
print(f"T-statistic: {t_stat:.5f}")
print(f"P-value: {p_value}")

```

T-statistic: 37.83196

P-value: 1.11782280577468e-309

The t-test results in a p-value of 1.11782280577468e-309, i.e. the unit price for mainstream, young and mid-age singles and couples are significantly higher than that of budget or premium, young and mid-age singles and couples.

```

[116]: segment1 = merged_data[
        (merged_data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
        (merged_data['PREMIUM_CUSTOMER'] == 'Mainstream')
    ]

other = merged_data[
    ~((merged_data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
        (merged_data['PREMIUM_CUSTOMER'] == 'Mainstream'))
]

quantity_segment1 = segment1['PROD_QTY'].sum()
quantity_other = other['PROD_QTY'].sum()

quantity_segment1_by_brand = segment1.groupby('BRAND').agg(
    targetSegment=('PROD_QTY', lambda x: x.sum() / quantity_segment1)
).reset_index()

quantity_other_by_brand = other.groupby('BRAND').agg(
    other=('PROD_QTY', lambda x: x.sum() / quantity_other)
).reset_index()

brand_proportions = pd.merge(quantity_segment1_by_brand,
↳quantity_other_by_brand, on='BRAND')

brand_proportions['affinityToBrand'] = brand_proportions['targetSegment'] /
↳brand_proportions['other']

```

```
brand_proportions = brand_proportions.sort_values(by='affinityToBrand',
↪ascending=False)

print(brand_proportions)
```

	BRAND	targetSegment	other	affinityToBrand
18	TYRRELLS	0.031553	0.025692	1.228095
17	TWISTIES	0.046184	0.037877	1.219319
5	DORITOS	0.122761	0.101075	1.214553
9	KETTLE	0.197985	0.165553	1.195897
16	TOSTITOS	0.045411	0.037978	1.195713
11	PRINGLES	0.119420	0.100635	1.186670
4	COBS	0.044638	0.039049	1.143124
8	INFUZIONI	0.064679	0.057065	1.133435
15	THINS	0.060373	0.056986	1.059423
7	GRNWVES	0.032712	0.031188	1.048873
3	CHEEZELS	0.017971	0.018647	0.963753
13	SMITHS	0.096370	0.124584	0.773536
6	FRENCH	0.003948	0.005758	0.685569
2	CHEETOS	0.008033	0.012067	0.665733
12	RRD	0.043810	0.067494	0.649091
10	NATURAL	0.019600	0.030854	0.635241
1	CCS	0.011180	0.018896	0.591677
14	SUNBITES	0.006349	0.012580	0.504698
19	WOOLWORTHS	0.024099	0.049427	0.487573
0	BURGER	0.002926	0.006596	0.443597

We can see that: - Mainstream young singles/couples are 23% more likely to purchase Tyrrells chips compared to the rest of the population.

- Mainstream youngsingles/couples are 56% less likely to purchase Burger Rings compared to the rest of the population

Let's also find out if our target segment tends to buy larger packs of chips.

2 Deep dive into Mainstream, young singles/couples

using Affinity Analysis

```
[117]: quantity_segment1_by_pack = segment1.groupby('PACK_SIZE').agg(
        targetSegment=('PROD_QTY', lambda x: x.sum() / quantity_segment1)
    ).reset_index()

quantity_other_by_pack = other.groupby('PACK_SIZE').agg(
        other=('PROD_QTY', lambda x: x.sum() / quantity_other)
    ).reset_index()
```

```

pack_proportions = pd.merge(quantity_segment1_by_pack, quantity_other_by_pack,
    ↪on='PACK_SIZE')

pack_proportions['affinityToPack'] = pack_proportions['targetSegment'] /
    ↪pack_proportions['other']

pack_proportions = pack_proportions.sort_values(by='affinityToPack',
    ↪ascending=False)

print(pack_proportions)

```

	PACK_SIZE	targetSegment	other	affinityToPack
17	270.0	0.031829	0.025096	1.268287
19	380.0	0.032160	0.025584	1.257030
18	330.0	0.061284	0.050162	1.221717
4	134.0	0.119420	0.100635	1.186670
2	110.0	0.106280	0.089791	1.183637
14	210.0	0.029124	0.025121	1.159318
5	135.0	0.014769	0.013075	1.129511
16	250.0	0.014355	0.012781	1.123166
9	170.0	0.080773	0.080986	0.997370
6	150.0	0.157598	0.163421	0.964372
10	175.0	0.254990	0.270007	0.944382
8	165.0	0.055652	0.062268	0.893757
12	190.0	0.007481	0.012442	0.601271
11	180.0	0.003589	0.006067	0.591538
7	160.0	0.006404	0.012373	0.517616
1	90.0	0.006349	0.012580	0.504698
3	125.0	0.003009	0.006037	0.498442
13	200.0	0.008972	0.018656	0.480899
0	70.0	0.003037	0.006322	0.480292
15	220.0	0.002926	0.006596	0.443597

It looks like Mainstream young singles/couples are 27% more likely to purchase a 270g pack of chips compared to the rest of the population but let's dive into what brands sell this pack size.

Let's also find out if our target segment tends to buy larger packs of chips.

```

[119]: unique_prod_names = merged_data[merged_data['PACK_SIZE'] == 270]['PROD_NAME'].
    ↪unique()
print(unique_prod_names)

```

```
['Twisties Cheese      270g' 'Twisties Chicken270g']
```

Twisties are the only brand offering 270g packs and so this may instead be reflecting a higher likelihood of purchasing Twisties.

3 Conclusion

Sales have mainly been due to Budget-older families, Mainstream-young singles/couples, and Mainstream- retirees shoppers. We found that the high spend in chips for mainstream young singles/couples and retirees is due to there being more of them than other buyers. Mainstream, midage and young singles and couples are also more likely to pay more per packet of chips. This is indicative of impulse buying behavior. We've also found that Mainstream young singles and couples are 23% more likely to purchase Tyrrells chips compared to the rest of the population. The Category Manager may want to increase the category's performance by off-locating some Tyrrells and smaller packs of chips in discretionary space near segments where young singles and couples frequent more often to increase visibility and impulse behavior. Quantum can help the Category Manager with recommendations of where these segments are and further help them with measuring the impact of the changed placement.